SopraFS26 - Codenames Online

Introduction

This project is a real-time multiplayer implementation of the board game *Codenames*. The goal is to allow groups to play together without having to carry the physical version with them: two players act as the *Spymasters* who give one-word clues, while the others act as *Spies* who guess the corresponding word cards on the board. The backend handles lobby management, game state (board generation, turn order, scoring), WebSocket broadcasting, and persistence of histories.

Motivation - To provide a fully online Codenames experience with a reactive UI, time limits and configurable settings. This is a semester project for the *Software Engineering Lab* at UZH.

Technologies Used

- **Java 17** - Core language
- **Spring Boot 3.x** - Application framework
- **Spring Data JPA** - Database persistence
- **Spring WebSocket** (with STOMP) - Real-time communication
- **H2** - Database (in-memory)
- **Gradle** - Build tool
- **JUnit 5, Mockito, Spring Boot Test** - Testing

High-Level Components

Components	Role	Main class / file
Lobby Management	Create, join, leave lobbies; transfer host; assign teams/roles; configure settings (time limits, rounds, difficulty).	`LobbyController` / `LobbyService`
Game Engine	Start game; generate board (25 words + card types); track scores; check win conditions; handle game restart and archiving.	`GameController` / `GameService`
Turn & Clue/Guess Logic	Process clues (word + number); handle guesses (reveal cards, update score, switch turns); enforce rules.	`TurnService`
WebSocket Messaging	Broadcast live lobby updates (leaving, role changes); Broadcast live game updates (board, clue, guess, turn change, timer) to different roles (spymaster vs spy).	`GameWebSocketHandler` / `LobbyWebSocketHandler`

Components	Role	Main class / file
Timer Service	Scheduled task (every second) that checks time limits for current turn and auto-ends turns when time runs out.	`TimerService`

WebSocket handlers use the services to modify state and then broadcast changes to all clients subscribed to a lobby.

```
src/main/java/ch/uzh/ifi/hase/soprafs26/
├── config/ # Security
├── constant/ # Enums (CardType, Role, TeamColor, ...)
├── controller/ # REST endpoints (Lobby, Game, Debug)
├── entity/ # JPA entities (Lobby, Game, Turn, Board, ...)
├── exceptions/ # Global exception handling
├── repository/ # Spring Data JPA repositories
├── service/ # Business logic (LobbyService, GameService, TurnService, ...)
├── util/ # Helper classes (LinkParser)
└── websocket/ # WebSocket config, handlers, events
```

Launch & Deployment

Prerequisites

- **JDK 17** (Or later)
- **Gradle** (Wrapper included - can be used via `./gradlew`)
- **Git** (To clone the repository)

Local Development

1. **Clone the repository**

```
git clone https://github.com/SolarisCulture/sopra-fs26-group-25-server.git
cd sopra-fs26-group-25-server
```

2. **Build and run tests**

```
./gradlew clean build
```

3. **Start the application**

```
./gradlew bootRun
```

The server will start on `http://localhost:8080`. The H2 database runs in-memory by default (data is lost after restart). You can verify the server is running by visiting `http://localhost:8080`. You should see `"The application is running."`

4. **Run only the tests**

```
./gradlew test
```

New releases are automatically built and deployed when changes are pushed to the `main` branch. You can additionally manually trigger a deployment by re-running the workflow in the *Actions* tab.

> **Note:** If you encounter missing dependencies after pulling new changes or switching branches, run `./gradlew clean build` to force a re-download.

Roadmap

Authors and acknowledgement

License